Evaluation Methodology

Language-model loss, functional tests, similarity, AI judges, and comparative rankings

Use evaluation methods whose assumptions fit the task. Token predictability, reference similarity, preference, and functional success measure different things and should not be treated as interchangeable.

Source: Chip Huyen, AI Engineering: Building Applications with Foundation Models, first edition (2025). Source: book pp. 113–158 / PDF p. 137. These are original study notes, not verbatim chapter reproductions. Tool names, prices, model comparisons, and research findings reflect the book's publication period unless stated otherwise.

01 Challenges of Evaluating Foundation Models

Source: book pp. 114–119 / PDF p. 138

Why evaluation becomes harder

Open-ended tasks have many valid responses, powerful models require specialized evaluators, internals and training data can be opaque, and benchmarks saturate or leak. General-purpose models also create the problem of discovering unknown capabilities and risks.

Word of mouth and a handful of appealing examples are not a repeatable evaluation process. Maintain representative tests, explicit rubrics, versioned configurations, and expert checks proportional to consequences. Evaluation is both risk reduction and a way to discover opportunities for improvement.

02 Understanding Language Modeling Metrics

Source: book pp. 118–125 / PDF p. 142

Entropy, cross-entropy, and KL divergence

Entropy measures expected information or uncertainty in a distribution. Cross-entropy measures the cost of predicting actual outcomes using a model's distribution. Lower loss means more probability assigned to observed tokens, not necessarily better user-facing answers.

$$H(P) = -\Sigma\ P(x)\ \log\ P(x)$$
$$H(P,Q) = -\Sigma\ P(x)\ \log\ Q(x)$$
$$H(P,Q) = H(P) + D_{KL}(P \,||\, Q)$$

The data's irreducible uncertainty and mismatch between learned and true distributions both contribute. KL divergence is non-negative. Specify log base: base 2 gives bits, natural log gives nats.

BPC, BPB, and correct normalization

Bits per character and bits per byte divide total negative log probability in bits by the number of characters or bytes in the original text. They help compare compression-like performance across tokenizations.

$$\text{Total bits} = -\Sigma\ \log_2\ Q(x_i \mid x_{<i})$$
$$\text{BPC} = \text{total bits} / \text{character count}$$
$$\text{BPB} = \text{total bits} / \text{byte count}$$

Example: a 6-bit average token loss with an average of two characters per token gives 3 BPC. If those characters are one UTF-8 byte each, BPB is also 3. Multibyte characters change the denominator. Count actual encoded bytes instead of assuming every character is an arbitrary fraction of a byte.

Perplexity: interpretation and limits

$$PPL = 2^{H(P,Q)} \text{ for bit loss, or } \exp(H(P,Q)) \text{ for nat loss}$$
$$PPL(x_{1:n}) = \exp[-(1/n)\Sigma\ \ln\ Q(x_i|x_{<i})]$$

Perplexity is the inverse geometric mean token probability. Under a uniform effective-choice interpretation, perplexity 4 resembles uncertainty among four equally likely options; it does not generally mean 25% task accuracy.

Compare the same data, tokenization, context setup, and loss convention. More structure or useful context often lowers perplexity. Post-training may improve helpfulness while worsening next-token loss. Low perplexity can flag familiar/contaminated data, high perplexity unusual data, but neither proves duplication, leakage, or poor quality without investigation.

03 Exact Evaluation

Source: book pp. 125–136 / PDF p. 149

Functional correctness and pass@k

Judge whether the required function succeeds: tests pass, a reservation is actually made, or a requested artifact meets requirements. Tests need sufficient coverage and safe execution environments; passing weak tests is not proof of full correctness.

pass@k asks whether at least one of k generated candidates solves a problem. More candidates cost more and do not describe the quality of a single unselected answer.

> For n sampled candidates with c correct:
> estimated pass@k = 1 − C(n−c,k) / C(n,k)
> Use only when n ≥ k and sampling assumptions fit.

Similarity measurements against reference data

Exact match is useful for constrained tasks, but can reject correct paraphrases. Naive substring matching can accept an answer that includes the right keyword inside a false statement. Normalize only what the task permits.

Lexical similarity: edit distance counts insertions, deletions, substitutions and sometimes transpositions; n-gram overlap compares token sequences. BLEU is precision-oriented with a brevity penalty, ROUGE is commonly recall-oriented, and METEOR includes richer matching. These scores depend on tokenization and reference quality. A correct alternate caption, translation, or program may score poorly, while a near-identical but false statement may score well.

Embeddings and semantic similarity

Embeddings represent text or other data as vectors. A useful embedding places examples similar in the target sense close together. Token, sentence, document, and multimodal embeddings are distinct choices; higher dimension alone does not ensure better utility.

> $\text{cosine}(a,b) = (a \cdot b) / (||a||\ ||b||)$

Cosine normally ranges from −1 to 1; dot product is equivalent to cosine only for normalized vectors. Similar topic does not imply entailment or factual agreement. Evaluate embeddings using the intended retrieval/ranking task as well as available embedding benchmarks. CLIP-style joint spaces support text-image comparison through learned modality-specific encoders and projections.

04 AI as a Judge

Source: book pp. 136–148 / PDF p. 160

Why use it, and how?

AI judges can scale subjective review, explain decisions, work without a fixed reference for some tasks, and reduce human annotation costs. Three patterns are standalone scoring, candidate-versus-reference comparison, and pairwise candidate preference.

Define the task, evaluation criterion, score meanings, and output format. Include representative scored examples to calibrate the rubric. Short discrete scales or pairwise decisions can be easier to apply consistently than fine-grained ratings. A label such as "faithfulness" is not standardized across tools; inspect the actual prompt and model.

Limitations, bias, privacy, and cost

Judges can be inconsistent, favor their own outputs, prefer the first answer, or reward verbosity rather than quality. Swap candidate order, allow ties/uncertainty, blind provenance when possible, and compare against expert judgments. Low temperature reduces some variance but is not a correctness guarantee.

Version the judge prompt, model, scoring schema, sampling, and preprocessing. A change in the judge can look like a change in the application. Evaluation can cost more than generation; consider asynchronous judging or risk-based samples. External judging needs the same privacy review as external generation.

What models can act as judges?

A larger general model is not always necessary. A smaller specialized judge may perform better on a narrowly defined criterion. **Reward models** score prompt-response pairs; **reference-based judges** assess agreement with a target; **preference models** choose among responses.

Self-critique is useful feedback but not independent validation. Combine deterministic tests, calibrated judges, and human review. Some tasks are easier to verify than generate, while others require expertise beyond the proposed judge.

---

05 Ranking Models with Comparative Evaluation

Source: book pp. 148–158 / PDF p. 172

Pointwise scores versus pairwise preferences

Pointwise evaluation scores each model independently. Comparative evaluation collects head-to-head judgments on common prompts and uses them to infer relative strength. Permit ties rather than forcing random choices.

A simple win rate is wins divided by comparisons, with an explicit convention for ties. Opponent strength and sample mix matter. Elo-style systems update ratings from outcomes; Bradley-Terry models estimate latent strengths from pairwise data. Ratings are properties of the tested population and protocol, not universal measures of intelligence.

Challenges and the future of comparative evaluation

All pairs among n models require n(n-1)/2 pairings before repetitions. Active matching can prioritize uncertain comparisons. Preferences may be non-transitive, and uneven prompt distributions or weak/malicious raters can distort results. Report uncertainty and sufficient match counts, not only a rank.

Public rankings help screen candidates but may not reflect your users. Relative preference cannot tell whether either model clears your minimum quality bar. Correctness questions should not be settled by unqualified preference votes. Side-by-side comparison differs from randomized A/B tests of real product outcomes; use them as complementary evidence.

Revision Check

1. When does perplexity fail as a proxy for application success?
2. Why can exact, lexical, and semantic metrics disagree?
3. What does pass@k measure that pass@1 does not?
4. How would you calibrate and version an AI judge?
5. Why does winning a ranking not establish production readiness?

Choose a measurement for each failure mode and validate the measurement itself. No single score replaces functional checks, representative data, and accountable judgment.

Book-based notes, separate from the earlier transcript-based notes. Coverage is indexed by section in coverage.md. Source links open the supplied book at the relevant PDF page. Detailed examples, citations, and complete experimental evidence remain in the book.